信号星河：PMUX 通路择

面向 Yosys 综合流程的多路选择器（PMUX）结构优化

作品类别：FPGA 创新设计竞赛 · 赛题二　多路选择器 PMUX 结构优化

摘要

本作品面向 FPGA 综合后端中多路选择器（PMUX/MUX）与算术网络的面积优化问题，在开源综合工具 Yosys 0.69 源码树中实现了一个静态编译进主程序的自定义 C++ 优化 pass，并在 synth_intel 流程内部显式调用；基线版与优化版使用同一条综合命令 synth_intel -family cycloneiv，脚本逐字相同，对照差异仅体现在综合器二进制本身。方案由三层通用优化手段配合构成。其一为高层结构优化 pass pmux_opt：在选择器尚未被技术映射展开前遍历高层网表，当各分支由同型、等宽、无符号的二元布尔单元（仅 $xor、$and、$or）驱动并共享同一公共操作数时，依据布尔分配律将其提出到选择器输出级，消除分支中重复的同类单元；不满足条件即安全放弃。其二为算术进位网络优化：改写 Cyclone IV 算术技术映射，宽度不小于 6 的加减单元改用逐位全加器行波进位，宽度小于 6 回退通用前缀树，进位端口契约保持不变。其三为在 abc9 默认 AIG 脚本中插入若干纯组合、保守的精简命令，全部用例共用同一脚本。同环境两臂对照下，九个用例逻辑单元（cycloneiv_lcell_comb，含取反）合计由 2422 降至 2339，总量下降 3.43%，逐例算术平均下降 2.52%，无一反增，触发器合计 323 个保持不变。形式等价性由 EQY 在 abc9 打包之后、专用单元映射之前的 $lut 与标准触发器网表上分区 SAT 证明，九例全部 PASS、退出码 0，共证明 345 个分区等价。两臂各例五次平均，综合时间合计比值 1.048，主进程峰值内存合计比值 0.998，均在 1.05 阈值内。客观而言，平均降幅与更高目标仍有差距；在穷尽高层安全候选、确认无可共享的跨单元公共结构后，本方案保留已验证安全的规则集合，未为凑降幅引入针对个例或可能反增的变换，规则通用性、逐例不反增、触发器不增与脚本统一均经两臂回归和形式验证确认。

关键词：Yosys；多路选择器；综合优化；技术映射；形式等价验证；EQY；Cyclone IV

第一部分　作品概述

1.1 功能与特性

本作品是一套嵌入 Yosys 0.69 综合后端、面向多路选择器与算术网络的结构优化方案，随主程序静态编译，不依赖动态插件加载。综合入口命令保持为统一的 synth_intel -family cycloneiv，对照脚本中除综合器二进制外不做任何改动。方案具备以下特性：

（1）作用于高层网表：自定义 pass 在 $pmux、$mux 尚未被技术映射展开之前执行，面向结构化的选择器形态做确定性等价变换。

（2）通用而非个例：规则同时覆盖 $pmux 与 $mux，对全部用例统一生效，不为单一用例编写特例规则。

（3）条件安全触发：仅当分支驱动同型、等宽、无符号且公共操作数不回环时才执行变换；$add、$sub 等不满足分配律的算子一律不套用，条件不满足时原样保留原结构。

（4）算术映射带门槛保护：宽位宽算术单元使用行波进位，窄位宽单元回退原通用映射，避免窄位宽进位结构在 4-LUT 打包后反增面积。

（5）脚本统一：abc9 精简脚本对全部用例使用同一份命令序列，不按网表形态切换命令组合。

1.2 应用领域

本方案可服务于以下几类场景：

（1）FPGA 与 ASIC 综合后端的组合逻辑面积优化，覆盖多路选择、one-hot 译码、流水线选择、交叉开关（crossbar）、嵌套 case 语句等选择器密集型设计。

（2）基于 Yosys 的二次开发与综合 pass 教学：自定义 pass 的注册、高层网表遍历、安全条件判定以及在综合流程中的显式调用方式，可作为开源综合工具扩展开发的参考样例。

（3）算术单元技术映射研究：不同位宽下进位网络（前缀树与行波进位）的面积取舍与门槛保护策略，可迁移到其他查找表架构的目标工艺映射。

（4）形式等价验证流程搭建：两臂综合、分区导出、miter 合并与逐分区 SAT 证明的整套闭环方法，可复用于其他综合优化改动的正确性回归。

方案所面向的选择器与加法器网络，广泛存在于数据通路、寄存器组读端口、状态机译码、总线切换与片上互联等电路结构中。

1.3 主要技术特点

（1）静态编译进主程序的自定义 pass：pmux_opt 作为 Yosys 内置 pass 参与编译，由 synth_intel.cc 显式调用，符合非插件、脚本相同、二进制不同的交付形态。

（2）同构布尔分支公共操作数提出（R1）：分支为 P_i op C（op ∈ {xor, and, or}）且共享 C 时，由 MUX(P_i op C) ≡ MUX(P_i) op C 将 C 移到输出级，仅保留一次该运算，消除重复同类单元。

（3）宽位宽 ALU 行波进位映射（路线 D）：通用 techmap 前加载 Cyclone IV 算术映射；Y_WIDTH ≥ 6 的 $alu 用逐位全加器行波进位，小于 6 经 _TECHMAP_FAIL_ 回退 Brent-Kung 前缀树，CI、BI、X、CO 契约不变。

（4）保守统一的 abc9 AIG 精简（b4）：默认脚本去重、反相后插入 resub、fraig、sweep 三段纯组合精简；会导致反馈环与时序用例反增的激进命令不进入统一脚本。

（5）同环境两臂对照与分层等价证明：两臂同源码、同编译器、同参数分别编译；等价性在 abc9 之后的 $lut 层以分区 SAT 证明。

1.4 主要性能指标

在九个官方用例、同环境两臂对照下，本方案的总体性能指标汇总如表 1 所示；逐例明细见第三部分。

表 1 总体性能指标汇总

指标	基线原版	优化版	比值 / 结论
逻辑单元合计（LUT）	2422	2339	总量下降 3.43%，逐例算术平均 2.52%
触发器合计（FF）	323	323	持平，未增加
形式等价验证	—	9/9 用例 PASS，rc=0，345 分区	全部证明等价
综合时间合计	7.346 s	7.698 s	时间比值 1.048（≤1.05）
主进程峰值内存合计	231.398 MB	230.825 MB	内存比值 0.998（≤1.05）

1.5 主要创新点

（1）面向高层选择器的同构布尔公共操作数确定性提出规则，并配套同型、等宽、无符号、非回环、算子白名单等一组安全放弃条件，使变换在保证功能等价的前提下尽量扩大覆盖面。

（2）针对 Cyclone IV 目标的宽位宽算术单元进位结构替换，并以位宽门槛保护窄位宽回退，兼顾不同宽度下的打包面积收益与反增风险。

（3）形成“高层结构变换—算术技术映射—保守 AIG 精简”三层配合的优化组合，并建立两臂回归、分区 SAT 等价证明与时间、内存效率测量的完整验证闭环。

1.6 设计流程

整体综合流程如图 1 所示：读取 Verilog 并完成层次检查后，先执行新增的 pmux_opt 高层结构优化，再依次加载 Cyclone IV 算术映射与通用技术映射，随后由 abc9 在 b4 脚本下完成 AIG 精简与 4-LUT 打包，最后映射为 Cyclone IV 专用单元并统计单元数量。基线臂与优化臂在同一流程、同一用例与参数下并行综合，结果逐例对照；等价性由独立的 EQY 回路证明。

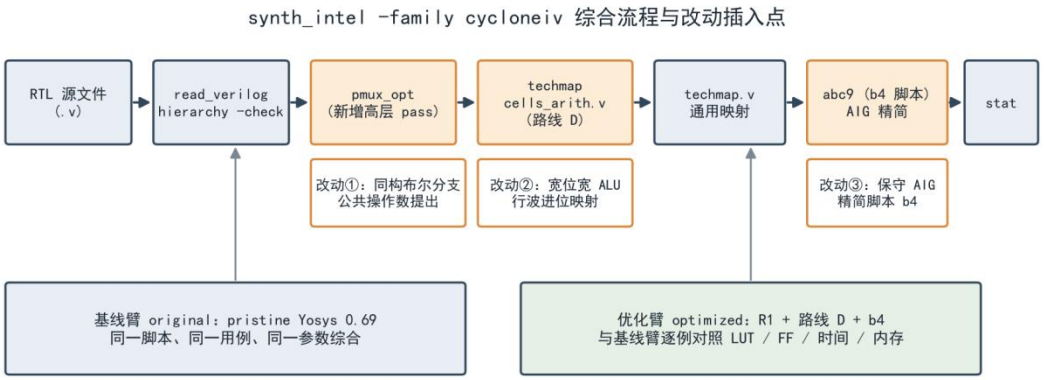


图 1 synth_intel 综合流程与三处改动插入点

第二部分　系统组成及功能说明

2.1 整体介绍

本系统由软件流程中的三类改动与一条独立的等价验证回路构成，整体关系如图 1 所示。三类改动分别是高层选择器优化 pass、算术单元技术映射文件与 abc9 内置精简脚本；它们沿综合流程先后衔接，前一处改动改变高层网表形态，后一处在不同层级继续收敛面积。两臂（未改动的原版 0.69 与本方案优化版 0.69）使用同一套综合脚本与用例，仅综合器二进制不同，以保证对照公平。等价验证回路独立于综合面积统计，从两臂 abc9 之后的中间网表出发完成形式证明。

2.2 硬件系统介绍

2.2.1 硬件整体介绍

本赛题为纯 EDA 软件综合优化任务，优化对象是综合工具内部的网表变换规则与技术映射文件，不涉及 FPGA 开发板、芯片上板、外接电路或机械结构，系统的全部组成单元为软件。

2.2.2 机械设计介绍

不适用。本作品无任何机械组件，不存在机械结构、装配与外壳设计。

2.3 软件系统介绍

2.3.1 软件整体介绍

全部改动位于 Yosys 0.69 源码树内部，静态编译进综合器主程序，不使用外部插件机制。改动在源码树中的分布如图 2 所示，共涉及四类源码与两个构建登记文件：

（1）passes/opt/pmux_opt.cc：新增 pmux_opt 高层优化 pass，实现 R1 公共操作数提出规则及其安全判定条件。

（2）techlibs/intel/synth_intel.cc：在通用 techmap 之前显式调用 pmux_opt 与 Cyclone IV 算术映射，相对原版仅增加两行调用语句。

（3）techlibs/intel/cycloneiv/cells_arith.v：路线 D 算术映射，宽位宽 $alu 行波进位与位宽门槛回退。

（4）passes/techmap/abc9.cc：将 default 内置脚本替换为 b4 保守 AIG 精简命令串。

（5）passes/opt/CMakeLists.txt 登记新 pass；techlibs/intel/CMakeLists.txt 登记新增的 cells_arith.v 数据文件。



图 2　Yosys 0.69 源码树改动映射与目录结构

2.3.2 软件各模块介绍

下面按综合流程先后顺序，分别介绍四个软件模块的输入、处理流程、输出与关键判定条件。

模块一：pmux_opt 高层选择器优化 pass。

输入为经过常规优化、尚未进行技术映射的高层 RTLIL 网表；输出为完成公共操作数提出后的等价网表。模块遍历网表中的 $pmux 与 $mux 单元，收集每个分支的驱动单元，依次校验同型、等宽、无符号、公共操作数不回环等条件，再判定分支算子是否属于 xor、and、or 白名单；任一条件不满足即安全放弃、保留原结构。判定通过后，将公共操作数 C 提出到选择器输出级，仅在输出端保留一次该布尔运算，消除其余分支中重复的同类单元。处理流程如图 3 所示。

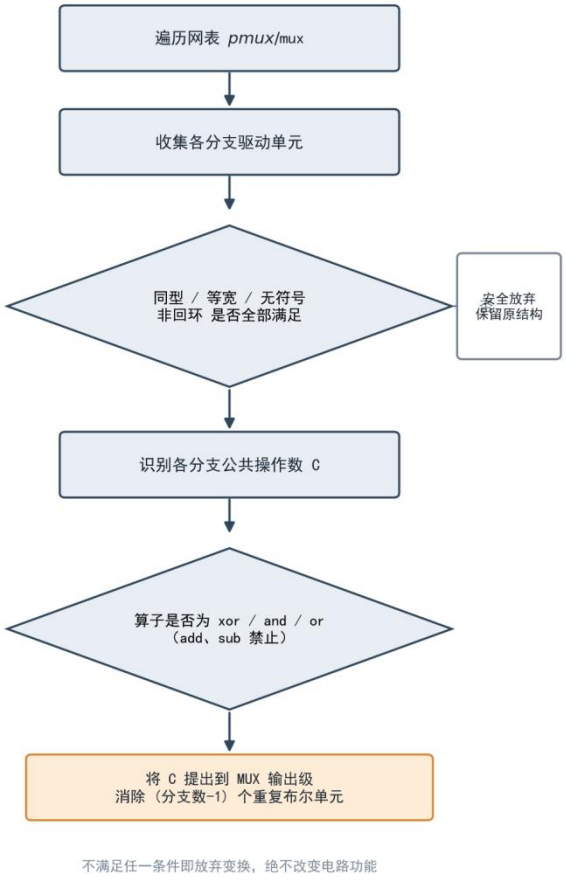


图 3　pmux_opt 规则判定与变换流程

模块二：Cyclone IV 算术单元技术映射（路线 D）。

输入为高层 $alu 算术单元；输出为映射后的门级或全加器网络。模块在通用 techmap 之前被 synth_intel 显式加载，命中 $alu 后先读取输出位宽 Y_WIDTH：当 Y_WIDTH ≥ 6 时，采用逐位 $fa 全加器级联的行波进位链，信号链短、单元规整；当 Y_WIDTH < 6 时，通过 _TECHMAP_FAIL_ 交还通用 Brent-Kung 前缀树映射。两种实现保持 CI、BI、X、CO 信号契约不变，功能等价。位宽分支决策如图 4 所示。

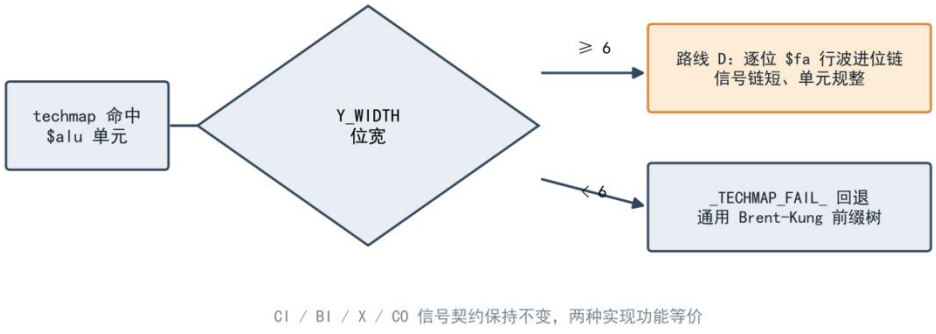


图 4　算术单元位宽分支决策流程

模块三：abc9 保守 AIG 精简脚本（b4）。

输入为 abc9 阶段的与反相图（AIG）；输出为精简后的 AIG，随后进入 4-LUT 打包。本模块不改动 abc9 的面积与快速模式选择，仅在默认命令串的去重、反相处理之后，插入 resub（布尔重写替换）、fraig（功能等价哈希）、sweep（冗余清扫）三段纯组合精简命令；命令序列对全部用例统一，不按网表形态切换。

模块四：EQY 形式等价验证回路。

输入为基线臂与优化臂在 abc9 打包之后、专用单元映射之前的 $lut 加标准内部触发器网表；输出为逐分区的 SAT 等价证明结果。回路跳过 ff_map 触发器映射、不使用 autoname，由 EQY 完成 miter 合并与默认自动分区，再以 sat 策略、深度 20 对每个分区独立证明；两臂触发器逐例保持一致。验证回路如图 5 所示。

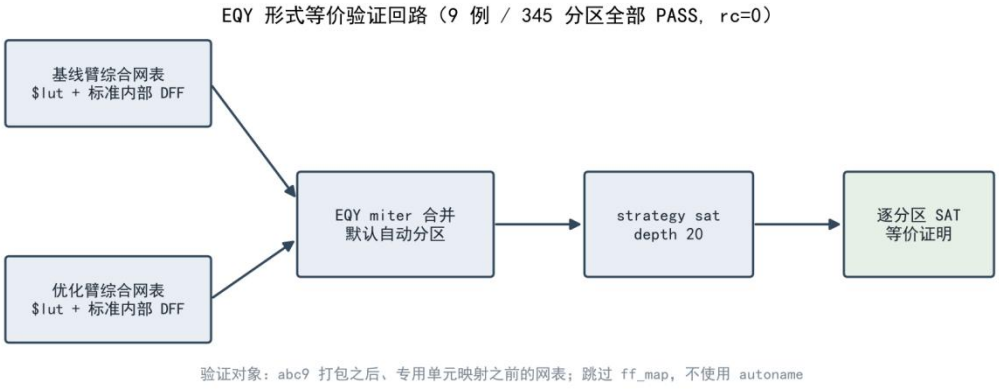


图 5　EQY 两臂 miter 分区 SAT 等价证明回路

第三部分　完成情况及性能参数

3.1 整体介绍

本作品无实物硬件，最终成果以两臂综合命令行运行结果、逐例单元对比表、EQY 证明日志与汇总图表呈现。全部测试在同一台主机、同一 Yosys 0.69 源码版本与同一构建参数下完成：基线臂为未改动的原版综合器，优化臂为本方案改动后的综合器；两臂使用相同的九个用例、相同的 synth_intel -family cycloneiv 命令与相同的运行目录，面积、等价性与效率分别独立测量并归档。

3.2 工程成果

3.2.1 机械成果

本作品为纯软件综合优化，无机械结构与实物样机。

3.2.3 软件成果

软件成果包括：编译进主程序的优化版 Yosys 0.69 综合器与未改动的基线版综合器、全部改动源码与统一 diff 补丁、参数化的综合对比、效率测量与 EQY 验证脚本、九个用例的两臂综合日志、EQY 等价证明归档，以及面积对比、效率测量与等价验证三份汇总结果文件。所有脚本使用相对路径与环境变量参数化，可在干净环境中按 README 步骤复现两臂构建、综合对照与等价证明。

3.3 特性成果

下面逐项展示功能实现情况与量化指标。面积统计对象为 Cyclone IV 逻辑单元 cycloneiv_lcell_comb（含取反），触发器统计对象为 dffeas；两臂逐例对照结果如表 2 所示。

表 2　九个用例两臂逻辑单元与触发器对照

用例	LUT 基线	LUT 优化	LUT 降幅	FF 基线	FF 优化
test1	286	286	0.00%	222	222
test2	25	25	0.00%	0	0
test3	12	12	0.00%	17	17
test4	35	34	2.86%	10	10
pmux_case01_balanced	426	392	7.98%	12	12
pmux_case02_pipeline	450	450	0.00%	24	24
pmux_case03_crossbar	408	373	8.58%	10	10

用例	LUT 基线	LUT 优化	LUT 降幅	FF 基线	FF 优化
pmux_case04_feedback	367	362	1.36%	16	16
pmux_case05_nested	413	405	1.94%	12	12
合计	2422	2339	总量 3.43% / 平均 2.52%	323	323

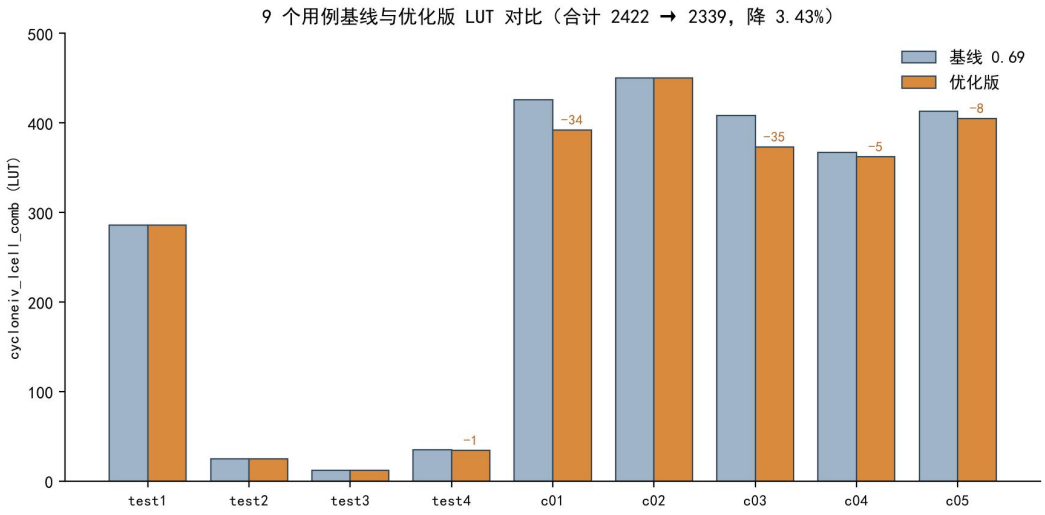


图 6　九个用例基线与优化版 LUT 对比

形式等价验证在 abc9 打包之后、专用单元映射之前的 $lut 与标准内部触发器网表上进行，采用默认自动分区与 sat 策略（深度 20），九个用例全部 PASS、退出码为 0，两臂触发器逐例一致，结果如表 3 所示。

表 3　EQY 分区 SAT 等价验证结果

用例	结果	退出码	证明分区数
test1	PASS	0	222
test2	PASS	0	5
test3	PASS	0	18
test4	PASS	0	10
pmux_case01_balanced	PASS	0	12
pmux_case02_pipeline	PASS	0	24
pmux_case03_crossbar	PASS	0	10
pmux_case04_feedback	PASS	0	32
pmux_case05_nested	PASS	0	12
合计	9/9 PASS	0	345

运行效率以两臂各例五次运行取平均测量：时间为整个 yosys 进程墙钟时间（含 abc

等待），内存为主进程峰值 RSS（不含 abc 子进程）；阈值为比值不超过 1.05，超过 1.05 进入警告区、超过 1.15 为红线。测量结果如表 4 所示。

表 4　两臂综合时间与内存效率对照（五次平均）

用例	时间基线 (s)	时间优化 (s)	时间比	内存基线 (MB)	内存优化 (MB)	内存比	判定
test1	0.824	0.858	1.041	27.011	27.155	1.005	OK
test2	0.630	0.656	1.041	21.463	21.526	1.003	OK
test3	0.684	0.694	1.015	22.205	22.003	0.991	OK
test4	0.704	0.730	1.037	22.599	22.723	1.005	OK
case01_balanced	0.906	0.942	1.040	28.197	27.316	0.969	OK
case02_pipeline	0.936	0.964	1.030	29.434	29.445	1.000	OK
case03_crossbar	0.914	0.984	1.077	27.159	27.036	0.995	警告
case04_feedback	0.844	0.916	1.085	26.246	26.631	1.015	警告
case05_nested	0.904	0.954	1.055	27.084	26.990	0.997	警告
合计	7.346	7.698	1.048	231.398	230.825	0.998	—

三个用例时间比值落在 1.05 至 1.15 的警告区间，但均未越过 1.15 红线；内存比值全部不超过 1.015，合计内存比值 0.998，未出现内存膨胀。

第四部分　总结

4.1 可扩展之处

（1）宽 PMUX 的 4-LUT 友好分解：当前以公共操作数提出为主，后续可对宽位选择器的两级、三级树状分解做定向优化，减少选择网络占用的查找表。

（2）跨通道译码复用：流水线与交叉开关用例中译码信号目前无法跨通道共享，可在更高层面对译码结构做公共子表达式识别。

（3）专用进位通路：本次算术映射停留在门层结构替换，尚未利用 Cyclone IV 专用进位输入通路，结合进位约束可进一步压缩宽数据通路面积。

（4）时序约束下的努力度调节：本次用例无时序约束，abc9 线延迟权重不参与面积决策；引入约束后可在面积与关键路径间重新平衡精简命令组合。

（5）规则算子与位宽扩展：在补充安全证明与全用例回归后，将公共操作数提出规则推广到更多满足分配律的算子组合与位宽场景。

4.2 心得体会

这次赛题是一次从凭经验猜优化点、走向用证据约束每一步改动的训练。最初我们直觉上认为多路选择器分支重复多，应当能获得明显面积下降；但把基线网表逐层拆开、对高层比较、算术与布尔单元做签名统计后才发现，多数重复结构在 Yosys 既有优化与 abc9 打包阶段已被合并，留给高层 pass 的安全空间比预想小得多。这让我们意识到，综合优化不能只看 RTL 表面写法，必须回到中间网表，按单元类型、位宽、驱动关系与回环情况逐一判断变换是否安全、是否真有收益。

规则设计上，我们把功能等价放在面积下降之前。公共操作数提出只允许 xor、and、or，把 add、sub 排除在外，正是因为后两类不满足分配律，直接提出会改变电路功能；同型、等宽、无符号、非回环等条件都是排查反例后补上的安全边界。算术进位网络的位宽门槛同样来自实测：放宽后窄位宽用例在 4-LUT 打包时反而反增，于是保留小于 6 位回退前缀树的保护。这些约束看似保守，却保证九例逻辑单元无一反增、触发器不增，也让结果经得起形式验证复核。

等价验证改变了我们对优化正确性的理解。面积变小只说明单元数量减少，不能证明功能未被破坏；EQY 在 abc9 之后的 $lut 层把两臂网表合并为 miter、自动分区并逐分区

SAT 证明，九例、三百四十五个分区全部通过、退出码 0。这种不依赖测试向量、对所有输入取值成立的证明方式，给高层变换提供了更硬的正确性依据；跳过触发器映射、不做 autoname，才能让两臂在可对应的层级完成分区证明。

效率测量让我们进一步克制优化冲动。新增 pass 与精简命令都会带来额外遍历与求解开销，因此每个候选改动都必须在同一环境五次取平均，记录墙钟时间与主进程峰值内存，对照 1.05 阈值判断是否保留。个别用例时间比进入警告区，说明高层精简与下游 abc9 之间存在重复劳动，也是后续可协调的方向；两臂同环境、同参数、脚本相同而仅二进制不同的对照方式，避免了把环境差异误当成优化收益。

客观地说，本方案平均降幅与更高面积目标之间仍有差距。我们尝试过共享多路选择器、跨通道译码复用、放宽进位门槛、更换更激进的 abc 重写命令等路线，均因反增、失败回退或缺乏安全证据而放弃，并在文档中如实记录，不为指标强行保留。这次经历让我们体会到，工程优化的价值不仅在于拿到多少百分比的下降，更在于每条保留下来的规则都能说清适用条件、安全边界与验证依据；工具链阅读、网表分析、形式验证与对照实验这些能力，也比单一面积数字更长期地留在手上。

第五部分　参考文献

[1] YosysHQ. Yosys Open SYnthesis Suite Documentation[EB/OL]. https://yosyshq.net/yosys/documentation.html.

[2] Wolf C. What is Yosys[EB/OL]. YosysHQ Read the Docs, https://yosyshq.readthedocs.io/projects/yosys/en/latest/introduction.html.

[3] Brayton R, Mishchenko A. ABC: An Academic Industrial-Strength Verification Tool[C]//International Conference on Computer Aided Verification (CAV 2010), LNCS 6174. Berlin: Springer, 2010: 24-40.

[4] Mishchenko A. ABC: A Simple System for Sequential Synthesis and Verification[EB/OL]. UC Berkeley, https://people.eecs.berkeley.edu/~alanmi/abc/.

[5] Mishchenko A. ABC: A Programmer's Manual[EB/OL]. https://people.eecs.berkeley.edu/~alanmi/abc/programming.pdf.

[6] YosysHQ. YosysHQ EQY Documentation[EB/OL]. https://yosyshq.readthedocs.io/projects/eqy/en/latest/.

[7] Engelhardt N. New Open Source Design Verification Tools from YosysHQ[C]//SEFUW 2023. ESA/ESTEC, 2023.

[8] Intel Corporation. Cyclone IV Featured Documentation and Device Handbook[EB/OL]. https://www.intel.cn/content/www/us/en/docs/programmable/767845/current/cyclone-iv-featured-documentation-quick.html.

[9] YosysHQ. Yosys Command Reference: techmap, abc9, miter[EB/OL]. https://yosyshq.readthedocs.io/projects/yosys/en/latest/cmd/index_passes_techmap.html.

[10] Chatterjee S, Mishchenko A, Brayton R. Factor Cuts[C]//IEEE/ACM International Conference on Computer-Aided Design (ICCAD 2006). San Jose: IEEE, 2006: 14-21.